

王树森强化学习

From蘑菇书+王树森：

核心目标：找到最佳策略 π （是个概率分布，具有随机性，避免可预判而失效），或者最佳动作价值函数 Q_{star}

基本概念

State space: set of $S(s_i)$ 状态

Action space: set of $A(a_i)$ 行为

Reward: 是数值，positive代表鼓励，negative表示惩罚。依赖于当前的state和action，而不是下一个state（eg.撞墙弹回原来位置和保持不动这两个action的下一个state都是原先位置，但要给前者惩罚）

Trajectory: (state – action – reward) chain

Return: 一个trajectory链上所有的reward总和

Episode: 从start到target1次就是1个episode

PDF概率密度函数: Policy $\pi(a|s)$

有两个随机性：action – $\pi(a|s)$ 决定； s' 状态转移函数 – $p(s'|a, s)$ 。第一个随机性来自agent采取策略的动作随机性，第二个随机性来自环境接受到agent动作后根据状态转移概率用随机抽样来得到下一个状态。

Discounted return: $U_t = R_t + \gamma \cdot R_{t+1} + \gamma^2 \cdot R_{t+2}^2$ ，每个R都是随机变量，取决于前一个a和p。

γ 是权重，距离最近的观测权重最高，介于0-1，是超参数，需要自己调。 R_i 取决于 A_i 和 S_i 。目标是让（长期） U_t 越大越好。

Action-Value function: $Q_{\pi}(s_t, a_t) = E[U_t | S_t = s_t, A_t = a_t]$ 和 π, s_t, a_t 都有关，是 U_t （未来所有奖励的加权求和）的条件期望，期望把未来的s和a都积分积掉，只留下 s_t 和 a_t 。

$Q^*(s_t, a_t) = \max_{\pi} Q_{\pi}(s_t, a_t)$ ：基于观测到的state，选择最佳action（ Q_{π} 给a动作打分）

State-Value function: $V_{\pi}(s) = E_A[Q_{\pi}(s_t, A)] = \sum_a \pi(a | s) \cdot Q_{\pi}(s, a)$

V_{π} 是 Q_{π} 的期望，评估当前局势，把A积分积掉，与动作a无关；

对于给定 π ，可以评估当前状态好坏；

通过积 s 求 V 期望，可以评估 π 的好坏

Markov decision process (MDP) :

M-马尔可夫性质-memoryless: 条件概率可以忽略历史，只关注 S_{t+1}, R_{t+1} 和 A_{t+1} ，未来只依赖于当前状态

d-就是Policy

p-就是几个possibility

Value-based价值学习

DQN (Deep Q-Network)

就是用神经网络近似 Q^* ，基于价值学习

Value network (critic) : $q(s, a; w)$ 来近似 $Q_\pi(s, a)$

$q(s, a; w) \rightarrow a^* = \operatorname{argmax} Q^*(s, a)$ 输入是 s ，输出是对每个 a 的打分， w 是神经网络的参数

TD算法 (Temporal Difference Learning)

(用来训练DQN，用奖励来更新模型参数)

观测到 s_t 和 a_t ,

预测到 $q_t = Q(s_t, a_t; w_t)$ ，输出 q_t 是对 a 的打分

用反向传播对DQN求导，得到梯度

环境提供新的 s_{t+1} 和 r_t

根据 Q 的意义是求 $E(U_t)$ ， U_t 的性质推得的

计算TD target: $y_t = r_t + \gamma \cdot q(s_{t+1}, a_{t+1}; w_t) = r_t + \gamma \cdot \max Q(s_{t+1}, a; w_t)$

$Loss = \frac{1}{2} \cdot [Q(s_t, a_t; w) - y_t]^2$ ，再做梯度下降减小loss

$Gradient : w_{t+1} = w_t - \alpha \cdot (q_t - y_t) \cdot d_t = w_t - \alpha \cdot \frac{\partial L(w)}{\partial w} \Big|_{w=w_t}$

Policy-based策略学习

Policy $\pi(a|s)$ 是概率密度函数，输入是 s ，输出是 a 的概率分布，agent进行随机抽样得到动作 a 。

Policy network (actor) :

$\pi(a|s; \theta)$ 来近似 $\pi(a|s)$, θ 是神经网络参数，一开始 θ 随机初始化，通过学习来更新 θ

$V(s; \theta) = \sum_a \pi(a|s; \theta) \cdot Q_\pi(s, a)$, 用来近似 $V_\pi(s_t)$, 更新 θ 使 V 变大

$J(\theta) = E_s[V(s; \theta)]$ 目标改进 θ , $J(\theta)$ 越大越好，用梯度上升

Policy gradient ascent

$$\theta \leftarrow \theta + \beta \cdot \frac{\partial V(s; \theta)}{\partial \theta}$$

$$Form1: \frac{\partial V(s; \theta)}{\partial \theta} = \sum_a \frac{\partial \pi(a|s; \theta)}{\partial \theta} \cdot Q_\pi(s, a)$$

$$Form2: \frac{\partial V(s; \theta)}{\partial \theta} = E_A \pi(\cdot|s; \theta) \left[\frac{\partial \log \pi(A|s; \theta)}{\partial \theta} \cdot Q_\pi(s, A) \right] \text{ 蒙特卡洛近似, 适用于连续情况}$$

Actor– Critic Method

State–Value function:

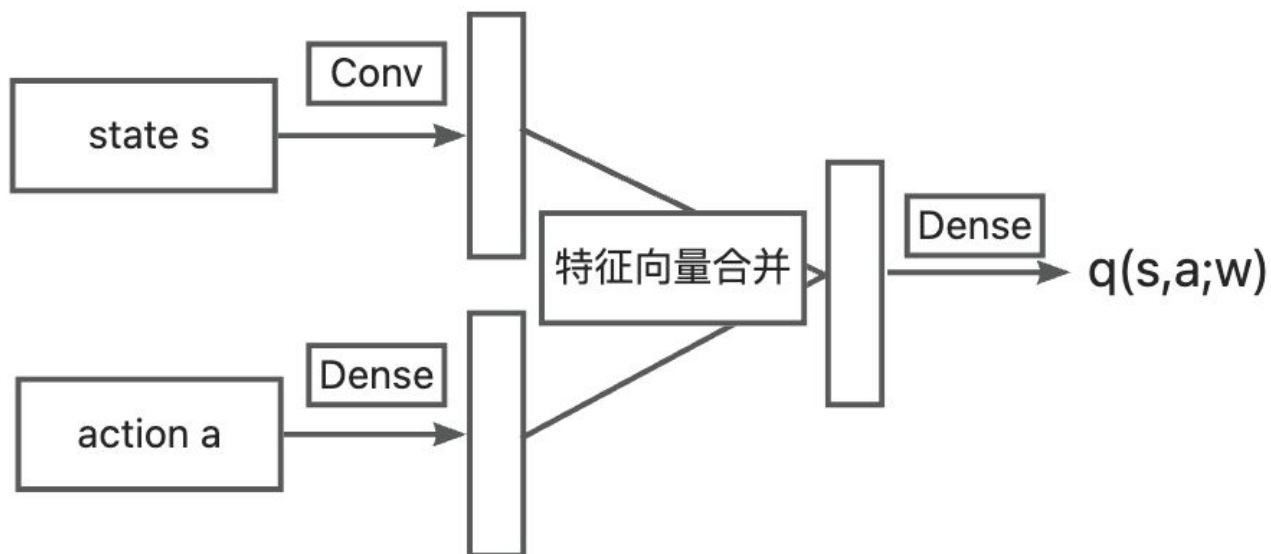
$$V_\pi(s) = \sum_a \pi(a | s) \cdot Q_\pi(s, a) \approx \sum_a \pi(a | s; \theta) \cdot q(s, a; w) \approx V(s; \theta, w)$$

思想：构造Value network和Policy network来近似两个函数，再用actor–critic方法同时学习两个网络

Policy network (actor) : $\pi(a|s; \theta)$ 来近似 $\pi(a|s)$, 策略控制运动，像运动员



Value network (critic) : $q(s, a; w)$ 来近似 $Q_\pi(s, a)$, 给运动打分，像裁判



Train:

- 更新policy network通过critic，即 θ ，为了提升V的大小，即提升打分
- 更新value network通过TD算法环境奖励反馈，即 w ，为了让打分更加精准

AlphaGo应用实例

首先是用behavior cloning训练策略，本质是模仿学习，和强化学习的区别在于不获得奖励反馈，只是模仿专家行为（16万局高段位对局棋谱）形成策略。局限点在于只能根据见过的state做出回应，没见过的state（招式）会表现的非常糟糕。进一步用强化学习训练可以弥补这个缺陷，因为会探索更多state，并且可以根据奖励知道应该做什么action。设置player和opponent，用policy gradient优化策略网络，再单独训练一个价值网络（先用策略网络做自我博弈）。实战用蒙特卡洛树搜索：策略函数排除不好的action，选出可能的action，再用策略函数假想对手的应招做模拟进行打分（包括评估state-value得分 v 和模拟下完整盘棋的 r ，然后把 $\frac{a + r}{2}$ 作为对a的打分，进一步更新action-values），选择得分最高的action

蒙特卡洛算法

随机算法：通过随机样本来估算真实值，不精确但够用

应用1:近似积分

一元函数：先随机抽 n 个样本，计算函数值，取平均，在乘以区间长度，记为 Q_n ，就是近似积分值。

Task: Given function $f(x) = \frac{1}{1+\sin(x) \cdot (\log_e x)^2}$, calculate $I = \int_{0.8}^3 f(x) dx$.

1. Draw n samples from $[0.8, 3]$ uniformly at random; denote them by $x_1, \dots, x_n$.
2. Calculate $Q_n = 2.2 \cdot \frac{1}{n} \sum_{i=1}^n f(x_i)$.
3. Return Q_n as an approximation to the integral $I = \int_{0.8}^3 f(x) dx$.

应用2:近似期望

X ：一维随机变量， $p(x)$ ：概率密度函数

先根据 $p(x)$ 随机抽样，计算函数值，取平均，记为 Q_n

Task: Estimate the expectation $\mathbb{E}_{X \sim p}[f(X)] = \int_{\mathbb{R}^d} f(x) \cdot p(x) dx$.

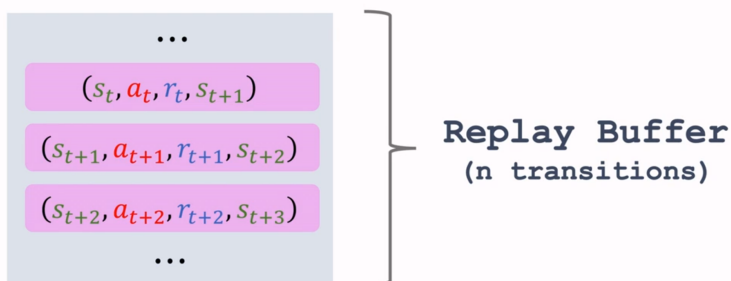
1. Draw n random samples from the probability distribution $p(x)$; denote them by $x_1, \dots, x_n$.
2. Calculate $Q_n = \frac{1}{n} \sum_{i=1}^n f(x_i)$.
3. Return Q_n as an approximation to $\mathbb{E}_{X \sim p}[f(X)]$.

Experience Replay

目标：重复利用之前的信息经验，并把信息经验打散消除相关性。提升DQN的表现的标准技巧

A transition: (s_t, a_t, r_t, s_{t+1}) 就是经验

Store recent n transitions in a **replay buffer**.



n 是个超参数，通常设置在百万级。如果存入Buffer数组的经验数超过 n ，就把最老的替换掉。

沿用TD算法，每次从buffer抽{mini-batch}个transition (s_t, a_t, r_t, s_{t+1}) ，计算TD error，SGD更新 w

Prioritized Experience Replay

思想：用非均匀抽样代替均匀抽样

少出现的transition更重要，一般用TD error判断，其绝对值越大（说明不熟悉），就更重要，给更大的权重（抽样概率）。

需要相应调整学习率，抵消掉不同权重带来的偏差（权重越大，学习率越小）

Transitions	Sampling Probabilities	Learning Rates
...	...	...
$(s_t, a_t, r_t, s_{t+1}), \delta_t$	$p_t \propto \delta_t + \epsilon$	$\alpha \cdot (n p_t)^{-\beta}$
$(s_{t+1}, a_{t+1}, r_{t+1}, s_{t+2}), \delta_{t+1}$	$p_{t+1} \propto \delta_{t+1} + \epsilon$	$\alpha \cdot (n p_{t+1})^{-\beta}$
$(s_{t+2}, a_{t+2}, r_{t+2}, s_{t+3}), \delta_{t+2}$	$p_{t+2} \propto \delta_{t+2} + \epsilon$	$\alpha \cdot (n p_{t+2})^{-\beta}$
...	...	...

Big $|\delta_t|$ ==> High probability ==> Small learning rate

高估问题

bootstrap: “自己把自己举起来”，用一个估算去更新其他估算（TD算法用 s_{t+1} 更新t时刻的估计）

高估原因: 1. TD公式由max操作（因为估值相当于随机增加均值为0的noise，最大估值的平均值肯定大于等于实际动作价值）。 2. 当前高估导致bootstrap更高估。

非均匀的动作价值高估会带来误导，做出非最佳动作。

Target Network

用另外一个target network代替TD target，避免自举，缓解bootstrap带来的高估问题（但不完全，因为 w^- 依赖于w）

- Target network: $Q(s, a; \mathbf{w}^-)$
 - The same network structure as the DQN, $Q(s, a; \mathbf{w})$.
 - Different parameters: $\mathbf{w}^- \neq \mathbf{w}$.
- Use $Q(s, a; \mathbf{w})$ to control the agent and collect experience:

$$\{(s_t, a_t, r_t, s_{t+1})\}.$$
- Use $Q(s, a; \mathbf{w}^-)$ to compute TD target:

$$y_t = r_t + \gamma \cdot \max_a Q(s_{t+1}, a; \mathbf{w}^-).$$
- Use a transition, (s_t, a_t, r_t, s_{t+1}) , to update $\mathbf{w}$.
 - TD target: $y_t = r_t + \gamma \cdot \max_a Q(s_{t+1}, a; \mathbf{w}^-)$.
 - TD error: $\delta_t = Q(s_t, a_t; \mathbf{w}) - y_t$.
 - SGD: $\mathbf{w} \leftarrow \mathbf{w} - \alpha \cdot \delta_t \cdot \frac{\partial Q(s_t, a_t; \mathbf{w})}{\partial \mathbf{w}}$.

Double DQN

缓解最大化造成的高估。关键在于selection和evaluation。让“选动作”和“评价值”分开，减少高估

- Selection using DQN: $a^* = \underset{a}{\operatorname{argmax}} Q(s_{t+1}, a; \mathbf{w})$.
- Evaluation using target network: $y_t = r_t + \gamma \cdot Q(s_{t+1}, a^*; \mathbf{w}^-)$.
- Double DQN alleviates overestimation:

$$\underbrace{Q(s_{t+1}, a^*; \mathbf{w}^-)}_{\text{Estimation by Double DQN}} \leq \underbrace{\max_a Q(s_{t+1}, a; \mathbf{w}^-)}_{\text{Estimation by target network}}$$

Dueling Network

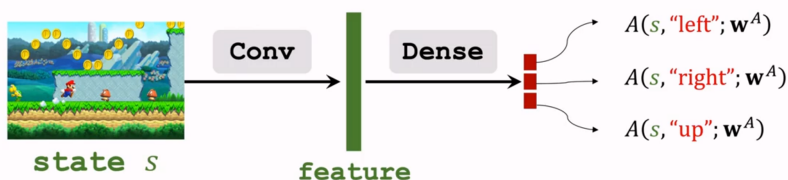
Optimal advantage function = Optimal action-value function – Optimal state-value function :

$$A^*(s, a) = Q^*(s, a) - V^*(s) \text{ 意义是动作} a \text{ 相对于baseline的优势}$$

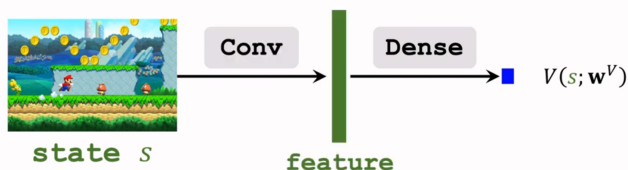
$$\text{根据 } V^*(s) = \max_a Q^*(s, a) \quad \max_a A^*(s, a) = \max_a Q^*(s, a) - V^*(s) = 0$$

变形得到 $Q^*(s, a) = A^*(s, a) + V^*(s) - \max_a A^*(s, a)$ 加上max这一项可以避免不唯一性值 (V和A的上下波动导致)

- Approximate $A^*(s, a)$ by a neural network, $A(s, a; \mathbf{w}^A)$.

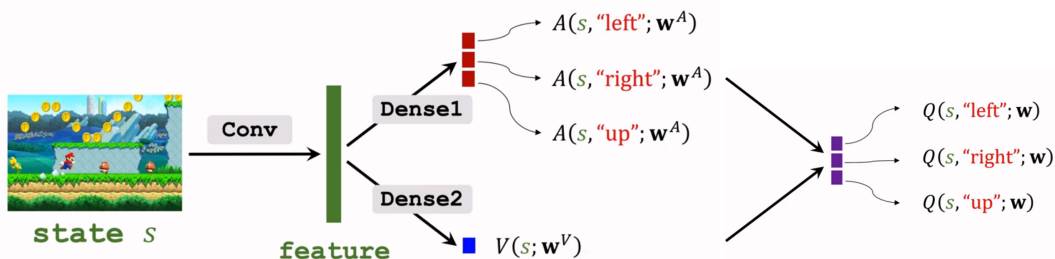


- Approximate $V^*(s)$ by a neural network, $V(s; \mathbf{w}^V)$.



由此可以用搭建的dueling network近似 $Q^*(s, a)$

$$Q(s, a; w^A, w^V) = V(s; w^V) + A(s, a; w^A) - \max_a A(s, a; w^A)$$



Training 继续用TD算法

策略梯度中的baseline